

Module - III

Sieve of Eratosthenes

Generate prime numbers with the help of an algorithm given by the Greek Mathematician named Eratosthenes, whose algorithm is known as Sieve of Eratosthenes.

It is a simple and oldest algorithm used to find the prime numbers up-to any given number.

For a given upper limit 'n', the algorithm works by marking the multiple of prime as starting from 2, when all multiples of 2 have been marked then the multiple of next prime i.e. 3 are marked. This process continues $p * p \leq n$.

Algorithm

1. To find all prime under 'n', construct a list of all numbers from 2 to n (1 is not prime).
2. Start with smallest prime number i.e. $P = 2$.
3. Mark all the multiple of P which are less than 'n'. To do this, we will mark the number as zero. (don't mark P itself).
4. Assign the value of P to the next non-zero number in the list.

5. Repeat the process $p * p \leq n$.

Example:

Fix all the number upto 25.

1. Construct a list upto 25, starting from 2.

$L = [2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25]$

2. Consider $p = 2$ and mark all the multiple of p as zero, but not p itself.

$L = [2, 3, 0, 5, 0, 7, 0, 9, 0, 11, 0, 13, 0, 15, 0, 17, 0, 19, 0, 21, 0, 23, 0, 25]$

3. Assign next non-zero value to p and mark all the multiple of p as zero, but not p itself.

$L = [2, 3, 0, 5, 0, 7, 0, 0, 0, 11, 0, 13, 0, 0, 0, 17, 0, 19, 0, 0, 0, 23, 0, 25]$

4. Repeat step 3 till $p * p \leq n$.

$L = [2, 3, 0, 5, 0, 7, 0, 0, 0, 11, 0, 13, 0, 0, 0, 17, 0, 19, 0, 0, 0, 23, 0, 0]$

5. For $p = 7$, $p * p \leq n$ is False. So we will not mark all the multiple of 7.

6. Therefore, the prime numbers are,

$L = [2, 3, 5, 7, 11, 13, 17, 19, 23]$.

Program Example:

```
n = int(input("Enter the upper limit: "))
```

```
prime = [True] * (n+1)
```

```
prime[0] = prime[1] = False
```

```
p = 2
```

```
while (p * p ≤ n):
```

```
    if prime[p]:
```

```
        for i in range(p * p, n+1, p):
```

```
            prime[i] = False
```

```
    p += 1
```

```
print("Prime Numbers Upto ", n)
```

```
print()
```

```
print([p for p, is_prime in enumerate(prime) if is_prime])
```

Output:

Enter the upper limit: 25

Prime Numbers Upto 25

[2, 3, 5, 7, 11, 13, 17, 19, 23]

Python File

- File are name of location on disk to store related information.
- When we want to read from or write to a file, we need to open it first.
- When we are done, it need to be closed. So that the resource that are tied with the file are free.
- The operation take place in following manner:
 - (i) Open a file
 - (ii) Read or Write (Perform operation)
 - (iii) Close the file.

Open Function:

The core function for file operation is `open()`, which take the file name and access mode as argument and return file object.

`file-object = open(file-name, access-mode)`

Example `file-object = open("file.txt", "r")`

File Modes:

The mode determined how the file is used.

- 'r' → Opens a file for reading only.
- 'w' → Opens a file for writing only.
- 'a' → Opens a file for appending.

'x' → Opens a file for exclusive creation.
'b' → Opens in binary mode.
't' → Opens a file for updating (reading & writing).

With Statement:

Using the with statement, is the way to handle file as it automatically ensure the file is close even if error occurs.

Example

```
with open("file.txt", "r") as file:  
    print(file.read())
```

Example for reading a file

```
file = open("file.txt", "r")  
data = file.read()  
print(data)  
file.close()
```

Example for writing to a file

```
file = open("file.txt", "w")  
file.write("Python")  
file.close()
```

Example for Reading File Line By Line

```
file = open("file.txt", "r")
```

```
for line in file:
```

```
    print(line)
```

```
file.close()
```

Example for Appending to File

```
file = open("file.txt", "a")
```

```
file.write("Python")
```

```
file.close()
```

`map()`, `filter()` & `reduce()` in Python

Python `map()`, `filter()` & `reduce()` are powerful tools that allow for efficient data transformation.

1. `map()`

The `map()` applies the specified function to each item of the loop.

Example:

```
list1 = [1, 2, 3, 4, 5]
```

```
num = list(map(lambda x: x*2, list1))
```

```
print(num) # [2, 4, 6, 8, 10]
```

2. `filter()`

It is useful for selecting specific items from a list based on certain criteria.

Example:

```
list1 = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

```
even_num = list(filter(lambda x: x%2 == 0, list1))
```

```
print(even_num) # [2, 4, 6, 8, 10]
```

3. reduce()

It apply a rolling computation to sequential pairs of value in a list.

Example:

```
from functools import reduce
```

```
list1 = [1, 2, 3, 4, 5]
```

```
num = reduce(lambda x, y: x+y, list1)
```

```
print(num)
```

Class

In Python, Class is a blueprint for creating object, that bundle data (attribute) and functionality (method) together.

Class are defined by using the "class" keyword.

Example:

```
class student:
```

```
    name = "Sandeep"
```

```
    section = "AI-IG"
```

```
    def display(self):
```

```
        print("My name is ", self.name)
```

```
        print("My section is ", self.section)
```

```
s = student()
```

```
print(s.display())
```

```
s.display()
```

Creating Object:

An object is created by calling the class using function notation. Attributes & Methods are access using ".".

Example:

```
class friend:
```

```
    name = input("Enter your friend name: ")  
    section = input("Enter your friend section: ")  
    marks = int(input("Enter your friend marks: "))
```

```
    def display(self):  
        print(self.name, self.section, self.marks)
```

```
f = friend()
```

```
f.display()
```

Special Method

Special method have double underscore starting and end of their name. It allow you to define how your object interact with built-in function and operators.

Some special methods are:

1. `--init-- (self, ...)`
2. `--str-- (self)`

`--init-- (self, ...):`

It runs automatically when a new object is created and is used to set the initial state of the object attribute.

Syntax:

```
def __init__(self, name):  
    self.name = name
```

```
__str__(self):
```

It defines the string representation of an object, which is called by the `print()` and `str()`.

Syntax:

```
def __str__(self):  
    return ....
```

Example:

```
class stu:
```

```
    def __init__(self, name):  
        self.name = name
```

```
    def __str__(self):  
        return f"Name: {self.name}"
```

```
s = stu("Sandeep")
```

```
print(s)          # Name: Sandeep
```

Comparison Method

These methods enable custom behavior for comparison operators. Some methods are:

`--eq--` (`==`), `--ne--` (`!=`), `--lt--` (`<`), `--gt--` (`>`),
`--le--` (`<=`) & `--ge--` (`>=`).

Example:

```
class Num:
```

```
    def __init__(self, value):  
        self.value = value
```

```
    def __gt__(self, other):  
        return self.value > other.value
```

```
n1 = Num(10)
```

```
n2 = Num(5)
```

```
print(n1 > n2) # True
```

Arithmetic Method

It enables operator overloading for mathematical operations, such as addition (`--add--` / `+`), subtraction (`--sub--` / `-`), & multiplication (`--mul--` / `*`) allow custom behaviour for class objects.

Example:

```
class Num:
    def __init__(self, value):
        self.value = value

    def __add__(self, other):
        return self.value + other.value
```

```
n1 = Num(10)
n2 = Num(20)
print(n1 + n2) # 30
```

Example:

```
class Num:
    def __init__(self, value):
        self.value = value

    def __str__(self):
        return f"Value is {self.value}"

    def __gt__(self, other):
        return self.value > other.value

    def __add__(self, other):
        return self.value + other.value
```

```
n1 = Num(30)
n2 = Num(20)
```

```
print(n1 + n2)      # 50
print(n1 > n2)      # True
print(n1)           # Value is 30
print(n2)           # Value is 20
```

String Methods/Functions

(i) len():

The `len()` returns the number of items in a object (list, tuple). When the object is string, it returns the number of characters in a string.

Example:

```
str1 = "Python Programming"
print(len(str1)) #
```

(ii) index():

First element in a sequence or string has index of zero, second element has an index of 1 and so on.

Example:

```
str1 = "This is classroom"
print(str1[4]) #
```

Indexing allows us to access the individual character in a string.

(iii) lower():

It converts a string into lower case.

Example:

```
str1 = "PyThon"  
print(str1.lower()) # python
```

(iv) upper():

It converts a string into upper case.

Example:

```
str1 = "Python"  
print(str1.upper()) # PYTHON
```

(v) title():

It converts a string to title case.

Example:

```
str1 = "Python programming"  
print(str1.title()) # Python Programming
```

(vi) swapcase():

It swap lower case character to upper case and upper case character to lower case.

Example:

```
str1 = "Python"  
print(str1.swapcase()) # pYTHON
```

(vii) capitalize():

It makes the first character in upper case and all other in lower case.

Example:

```
str1 = "this is classroom"
print(str1.capitalize()) # This is classroom
```

(viii) find():

It search the string for specified value and return the position of where it was found.

Example:

```
str1 = "Python Programming"
print(str1.find('n')) # 5
```

(ix) split():

It splits the string at the specified operator and returns an list.

Example:

```
str1 = "This is classroom"
print(str1.split('s')) # ['Thi', ' i', ' cla', '', 'room']
```

(x) type():

It returns the type of object or variable.

Example:

```
str1 = "Python Programming"  
print(type(str1)) # <class 'str'>
```

(xi) replace():

It return a string where a specified value is replaced with ~~the~~ a another specified value.

Example:

```
str1 = "This is classroom"  
print(str1.replace('s', 'z')) # This is clazzroom
```

Object Oriented Programming (OOP)

Object Oriented Programming is a programming style where we use class and object to structure code.

The main concept of OOPs are:

- Class
- Object
- Inheritance
- Encapsulation
- Polymorphism

Inheritance

Inheritance is a core principle of OOP that allow a new class to derive attributes and methods from an existing class. This method promotes code reusability, modularity and the modeling of real world "is-a" relationships.

For example: - A dog is an animal.

Example:

```
class Parent:  
    def show(self):  
        print("This is parent class.")
```

```
class Children(Parent):  
    def display(self):  
        print("This is child class.")
```

```
obj = Children()
```

```
obj.show() # This is parent class.
```

```
obj.display() # This is child class.
```

Types of Inheritance:

1. Single Inheritance:

When a class inherits from a single parent class.

Example:

```
class Father:  
    def show(self):  
        print("Round face")  
        print("Height: 6' 6'")
```

```
class Child(Father):  
    def display(self):  
        print("Face: Similar")
```

```
obj = Child()
```

```
obj.show() # Round Face  
           # Height: 6'
```

```
obj.display() # Face: Similar
```

2. Multi-level Inheritance:

A chain of inheritance where a class inherits from a derived class.

Example:

```
class Grandparent:
    def display1(self):
        print("Grandparent Class")
```

```
class Parent(Grandparent):
    def display2(self):
        print("Parent Class")
```

```
class Child(Parent):
    def display3(self):
        print("Child Class")
```

```
obj = Child()
```

```
obj.display1()    # Grandparent Class
obj.display2()    # Parent Class
obj.display3()    # Child Class
```

3. Multiple Inheritance:

When a class inherits from multiple parent classes.

Example:

```
class Mother:  
    def display1(self):  
        print("Brown Eyes")
```

```
class Father:  
    def display2(self):  
        print("Round Face")
```

```
class Child(Mother, Father):  
    def display3(self):  
        print("Brown eyes, Round face & Chubby Cheeks")
```

```
obj = Child()
```

```
obj.display1()    # Brown Eyes
```

```
obj.display2()    # Round Face
```

```
obj.display3()    # Brown eyes, Round face & Chubby Cheeks
```

Method Overriding

A child class can re-define a method that is already present in its parent class. When the method is called, then the child implementation will overwrite the parent.

Example:

class

class Parent:

def show(self):

print("Parent Class")

class Child(Parent):

def show(self):

print("Child Class")

obj = Child()

obj.show() # Child Class

Encapsulation

It involved bundling data and method within a single class to control ~~and~~ access and high detail.

It involved the following:

- (i) Bundling: Combining data and method that operate on data into a single unit which is the class.
- (ii) Data Hiding: It restricts direct access to sensitive data from the outside of the class to prevent modification or unauthorized access.
- (iii) Access Control: Python does not have strict keyword like `private` or `public`, but use naming convention as access modifier.

1. Public - This member can be access from anywhere define without any underscore prefix.

Example:- `self.name`

2. Protected - This member can be used within the class and its sub-class. Protected member can be define as `self._name`.

3. Private - It can be accessed within the class where they are define. It can be define by `self.__name`.

(iv) Getter/Setter Method: Controlled access to private and protected data is provided by public method (getter or setter) which can include validation logic.

Example:

```
class Student:
    def __init__(self, name, roll, marks):
        self.name = name
        self._roll = roll
        self.__marks = marks

    def get_marks(self):
        return self.__marks

    def set_marks(self, new_marks):
        if new_marks >= 0 and new_marks <= 100:
            self.__marks = new_marks
        else:
            print("Invalid Marks")

    def display(self):
        print("Name: ", self.name)
        print("Roll No.: ", self._roll)
        print("Marks: ", self.__marks)

s1 = Student("Sandeep", 37, 94)
print(s1.name)      # Sandeep
print(s1._roll)     # 37
print(s1.get_marks()) # 94
```

```
sl.set_marks(96)
```

```
sl.display()    # Name: Sandeep  
                # Roll No.: 37  
                # Marks: 96
```

Polymorphism

It means "many forms" that allow a single function, method or operator to behave different depending on the datatype or class of the object it is acting upon.

For Example:

```
def add(a, b):  
    return a + b
```

```
print(add(2, 3))    # 5
```

```
print(add("Hello", "Python")) # HelloPython
```

Assert Statement

The 'assert' is useful to ensure that a given condition is true. If it is not true, it raise AssertionError. If the condition is false, then the exception by the name AssertionError is raised along with the message.

If the message is not given and the condition is false then the assertion error is raised without message.

The 'assert' statement is useful to check if the given condition is fulfilled or not.

Example:

```
x = int(input("Enter number: "))
```

```
assert x > 0
```

```
print("Positive Number")
```

Exception Handling

An exception is an event, which occurs during the execution of a program that disturbs the normal flow of the program - ~~st~~ instruction.

It is a Python object that represents an Error.

Handling Exception:

If we have suspicious code that may raise an exception, then we can protect the program by placing the suspicious code in 'try' block.

Types of Exception:

1. ZeroDivision Error:

```
a = int(input("Enter no.: ")) # Enter no.: 2
b = int(input("Enter no.: ")) # Enter no.: 0
print(a / b) # ZeroDivisionError: division by zero
```

2. Name Error:

```
a = 5
print(b) # NameError: name 'b' is not defined
```

3. Type Error:

```
print('10' + 10) # TypeError
```

4. Value Error:

```
print(int('ab')) # ValueError
```

5. Index Error:

```
list1 = [1, 2, 3]  
print(list1[5]) # IndexError
```

6. Key Error:

```
data = {'Name': 'Sandeep'}  
print(data['age']) # KeyError: 'age'
```

Note: These exceptions are handled by `try... except... else... finally` blocks.

Example:

```
try:  
    a = int(input("Enter no.: "))  
    b = int(input("Enter no.: "))  
    print(a / b)  
except ZeroDivisionError:  
    print("Can not divide by zero.")  
else:  
    print("No error occurred.")  
finally:  
    print("Finish")
```

Importing Module in Python

In Python, the 'import' statement is used to bring the code from one module to another, it allows for code reuse and better organization.

We use modules for reusability, organized program and easy maintenance

Example:

→ Import complete module

```
import math
```

```
print(math.sqrt(16)) # 4.0
```

→ Import Specific Function

```
from math import sqrt  
print(sqrt(16)) # 4.0
```

→ Import All

```
from math import *  
print(sqrt(16)) # 4.0
```

→ Import Multiple Function

```
from math import sqrt, pi  
print(sqrt(16), pi) # 4.0 3.141592653589793
```

myname.py

```
def my_name(name):  
    print("Hello, my name is", + name)
```

main.py

```
import myname
```

```
myname.my_name("Sandeep")  
# Hello, my name is Sandeep
```

Abstract Data Type (ADT)

- An abstract data type is a datatype that is defined by its behaviour (operation, not by its implementation).
- It tells what to do, not how to do it.
- Examples of ADT's are: Stack, Queue, List, Set and Dictionary.
- ADT components are:

(i) Data:

The value stored in the structure.

(ii) Operation:

Function that can be performed on data.

→ ADT operations are:

push() to insert an element

pop() to delete an element.

peek() to view the top element.

ADT Interface in Python:

The ADT interface define what operation are available and how user interact with the ADT.

Note: It hides the internal implementation (data hiding).

Features of ADI:

- Encapsulation
- Data hiding
- Abstraction
- Clear Interface

Advantages of ADT:

- It improves the code clarity.
- Easy to modify implementation.
- Reusable and scalable.
- It reduce the complexity.

Example:

```
class Stack:
```

```
    def __init__(self):
```

```
        self.item = []
```

```
    def push(self, element):
```

```
        self.item.append(element)
```

```
        print(element, " Push")
```

```
    def pop(self):
```

```
        if not self.item:
```

```
            print("Stack is Empty")
```

else:

print(self.item.pop(), " Pop ")

def peek(self):

if self.item == []:

print("Stack is Empty")

else:

print("Top Element: ", self.item[-1])

def is_empty(self):

return len(self.item) == 0

obj = Stack()

obj.push(10) # 10 Push

obj.push(20) # 20 Push

obj.push(30) # 30 Push

obj.push(50) # 50 Push

obj.pop() # 50 Pop

obj.peak() # Top Element: 30

print(obj.is_empty()) # False

List ADT

It contains elements of same type arranged in sequential order. Methods:-

get() - Returns an element from a list at any given position.

insert() - Inserts a new element at any position of the list.

remove() - It removes the first occurrence of the given element from a non-empty list.

remove_at() - Remove the element at a specified location from a non-empty list.

size() - It returns the number of elements in list.

is_empty() - It returns True if the list is empty otherwise returns False.

is_full() - It returns True if the list is full otherwise returns False.

Example:

```
class List:
```

```
    def __init__(self, size_max):  
        self.items = []  
        self.max_size = size_max
```

```
    def get(self, pos):  
        if self.is_empty():  
            print("List is Empty")  
        else:  
            try:  
                print("Element at ", pos, "is", self.  
                    items[pos])  
            except IndexError:  
                print("The given index is out of range.")
```

```
    def insert(self, new_element, pos):  
        if self.is_full():  
            print("List is Full")  
        else:  
            self.items.insert(pos, new_element)  
            print(new_element, "Added")
```

```
    def remove(self, element):  
        if self.is_empty():  
            print("List is Empty")  
        else:  
            try:  
                self.items.remove(element)  
                print(element, "Removed")
```

```
except ValueError:  
    print("Element not in the List")
```

```
def remove_at(self, pos):  
    if self.is_empty():  
        print("List is Empty")  
    else:  
        try:  
            print(self.items.pop(pos), "Removed")  
        except IndexError:  
            print("Index out of range")
```

```
def size(self):  
    return len(self.items)
```

```
def is_empty(self):  
    return len(self.items) == 0
```

```
def is_full(self):  
    return len(self.items) == self.max_size
```

```
obj = List(10)
```

```
obj.insert(10, 0)    # 10 Added
```

```
obj.insert(20, 1)    # 20 Added
```

```
obj.insert(2, 0)     # 2 Added
```

```
obj.insert(30, 4)    # 30 Added
```

```
obj.get(0)           # Element at 0 is 2
```

```
obj.remove(30)        # 30 Removed
```

```
obj.remove_at(0)     # 2 Removed
```